Diagrama de Classe

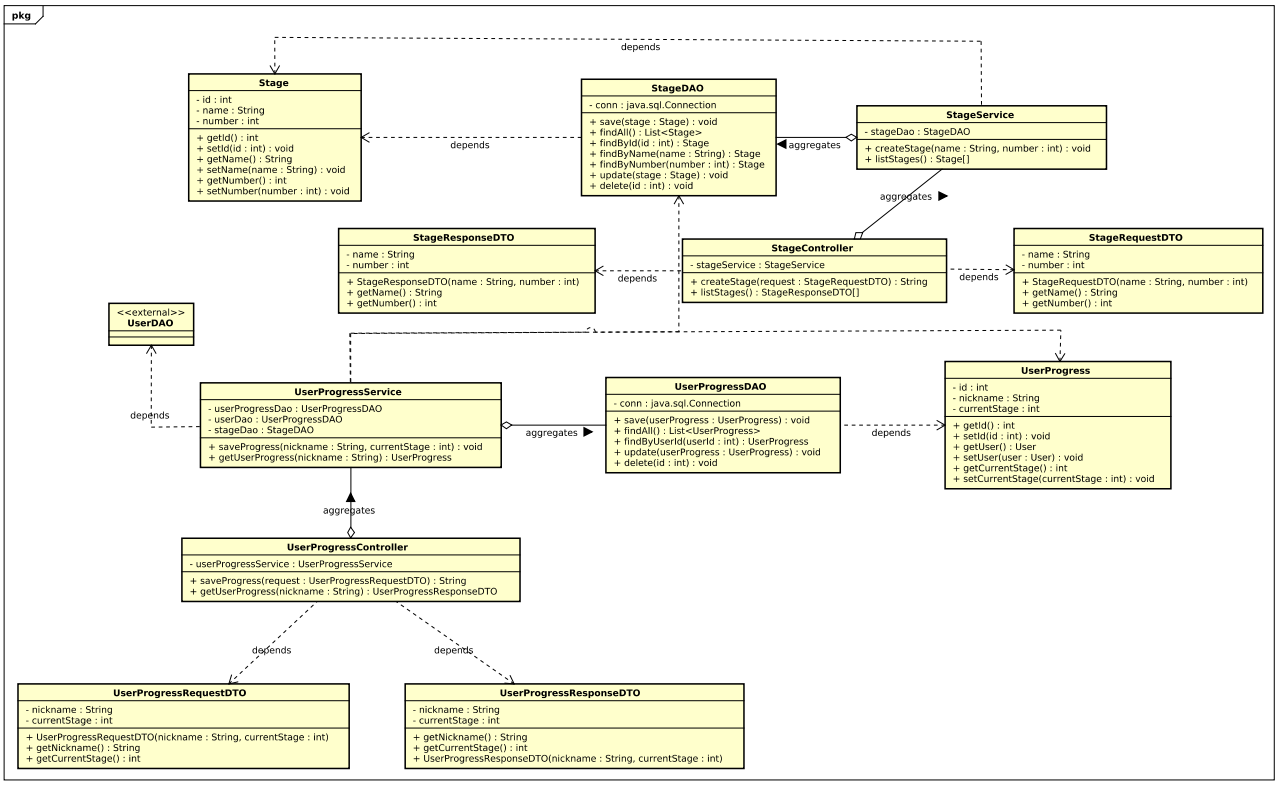
CSU02: Acessar lista de fases

Autor: Altamiro Santos Moura Belezia (Designer de Software)

Descrição do caso de uso:

Este caso de uso permite criar estágios dentro do sistema e acompanhar o progresso dos usuários nesses estágios. O usuário pode ter seu avanço salvo e recuperado, garantindo que ele continue de onde parou em sua experiência.

Diagrama:



Descrição do diagrama: O sistema é estruturado em camadas, cada uma com uma responsabilidade bem definida.

- **Entidades principais:**
  - A classe `Stage` representa um estágio, contendo informações como identificador, nome и número.
  - A classe `UserProgress` representa o progresso de um usuário, com atributos como identificador, apelido (`nickname`) e o estágio atual (`currentStage`).
- **Persistência:**

    - A classe `StageDAO` é responsável por todas as operações de banco de dados relacionadas a estágios (criação, busca, atualização e exclusão).
    - A classe `UserProgressDAO` desempenha o mesmo papel, mas para o progresso dos usuários.
    - A interface `UserDAO` define um contrato para acesso a dados de usuários, abstraindo a fonte de dados. Ela é utilizada pelo `UserProgressService` para obter informações relacionadas ao usuário.
- **Serviços:**
    - A classe `StageService` encapsula a lógica de negócio referente a estágios, permitindo criar e listar estágios. Ela agrega uma instância de `StageDAO`.
    - A classe `UserProgressService` centraliza a lógica de progresso dos usuários, permitindo salvar e consultar o estágio atual. Ele interage com múltiplos DAOs, agregando `UserProgressDAO` e `StageDAO`, e dependendo da interface `UserDAO` para validar ou buscar dados do usuário.
- **Controladores:**
    - A classe `StageController` atua como a interface para criar e listar estágios, recebendo requisições e repassando-as ao `StageService`.
    - A classe `UserProgressController` expõe métodos para salvar e consultar o progresso de um usuário, utilizando o `UserProgressService`.
- **Objetos de Transferência de Dados (DTOs):**
    - O sistema utiliza um padrão que separa os DTOs de entrada (Request) e saída (Response).
    - As classes `StageRequestDTO` e `UserProgressRequestDTO` são usadas para encapsular os dados enviados para os controladores.
    - As classes `StageResponseDTO` e `UserProgressResponseDTO` são usadas para formatar e enviar os dados de resposta do sistema, expondo apenas as informações necessárias.

Implementação: Para implementar o sistema, recomenda-se a seguinte abordagem em camadas:

1. **Modelagem das entidades:** Crie as classes `Stage` e `UserProgress`, representando os dados centrais.
2. **Camada de persistência:** Implemente `StageDAO` e `UserProgressDAO` para interagir com o banco de dados. Crie também a interface `UserDAO` para definir o contrato de acesso aos dados de usuário.
3. **Regras de negócio:** Crie `StageService` e `UserProgressService`, onde ficam as validações e a lógica de negócio (por exemplo, verificar se um estágio já existe ou associar o progresso a um usuário válido, utilizando as dependências `UserProgressDAO`, `StageDAO` e `UserDAO`).

4. **Camada de controle:** Construa `StageController` e `UserProgressController`, que recebem requisições externas (usando os `RequestDTOs`), processam os dados e acionam os serviços adequados.
5. **Transferência de dados:** Utilize `StageRequestDTO` e `UserProgressRequestDTO` para receber dados na camada de controle, e `StageResponseDTO` e `UserProgressResponseDTO` para expor apenas as informações necessárias como resposta, evitando expor detalhes internos das entidades.